

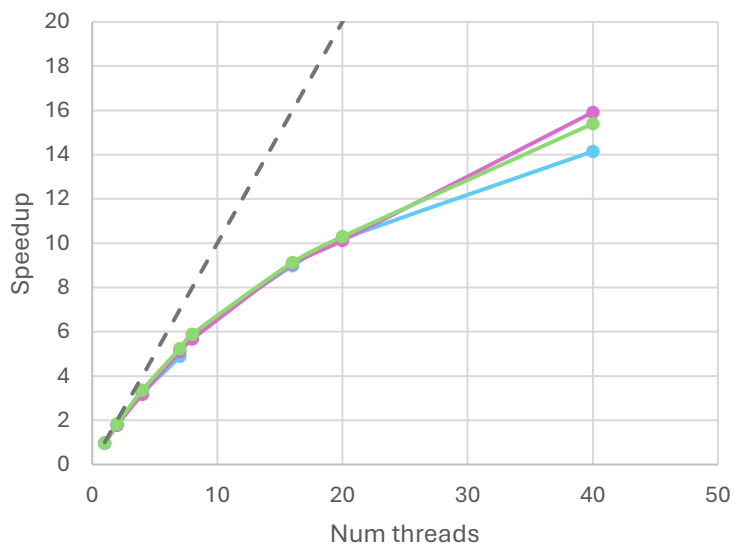
Спецификация

Server name	ProLiant XL270d Gen10
CPU	Intel(R) Xeon(R) Gold 6248 CPU @ 2.50GHz
Physical cores	20 cores
L1d / L1i	1.3 MiB × 40 instances each
L2	40 MiB × 40 instances
L3	55 MiB (2 instances)
NUMA nodes	2
RAM (node0 node1)	385636 MB 387008 MB
OS	Ubuntu 22.04.5 LTS

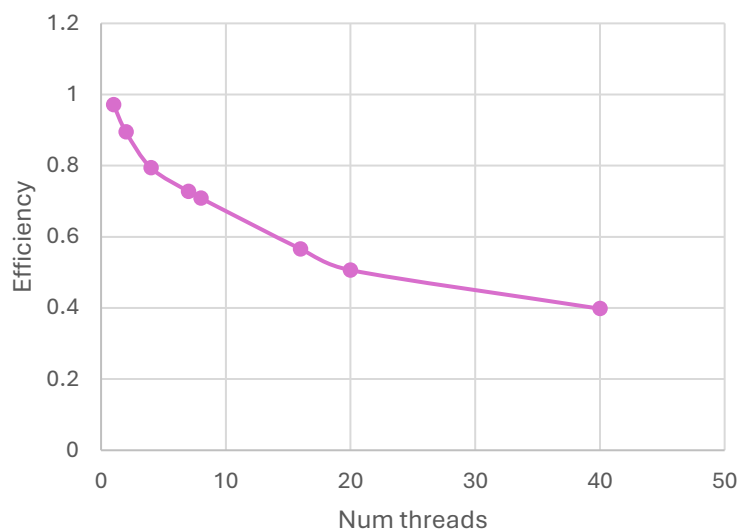
Результаты

Bind method (N= 4.0E7)	Thread amount														
	2			4		7		8		16		20		40	
	T1	T2	S2	T4	S4	T7	S7	T8	S8	T16	S16	T20	S20	T40	S40
Close	0.293	0.161	1.785	0.088	3.226	0.058	4.888	0.048	5.862	0.032	8.979	0.027	10.186	0.025	14.150
Spread	0.288	0.163	1.789	0.093	3.175	0.057	5.093	0.050	5.668	0.031	9.054	0.027	10.128	0.019	15.916
False	0.285	0.157	1.838	0.086	3.352	0.053	5.243	0.049	5.894	0.031	9.129	0.027	10.300	0.019	15.402

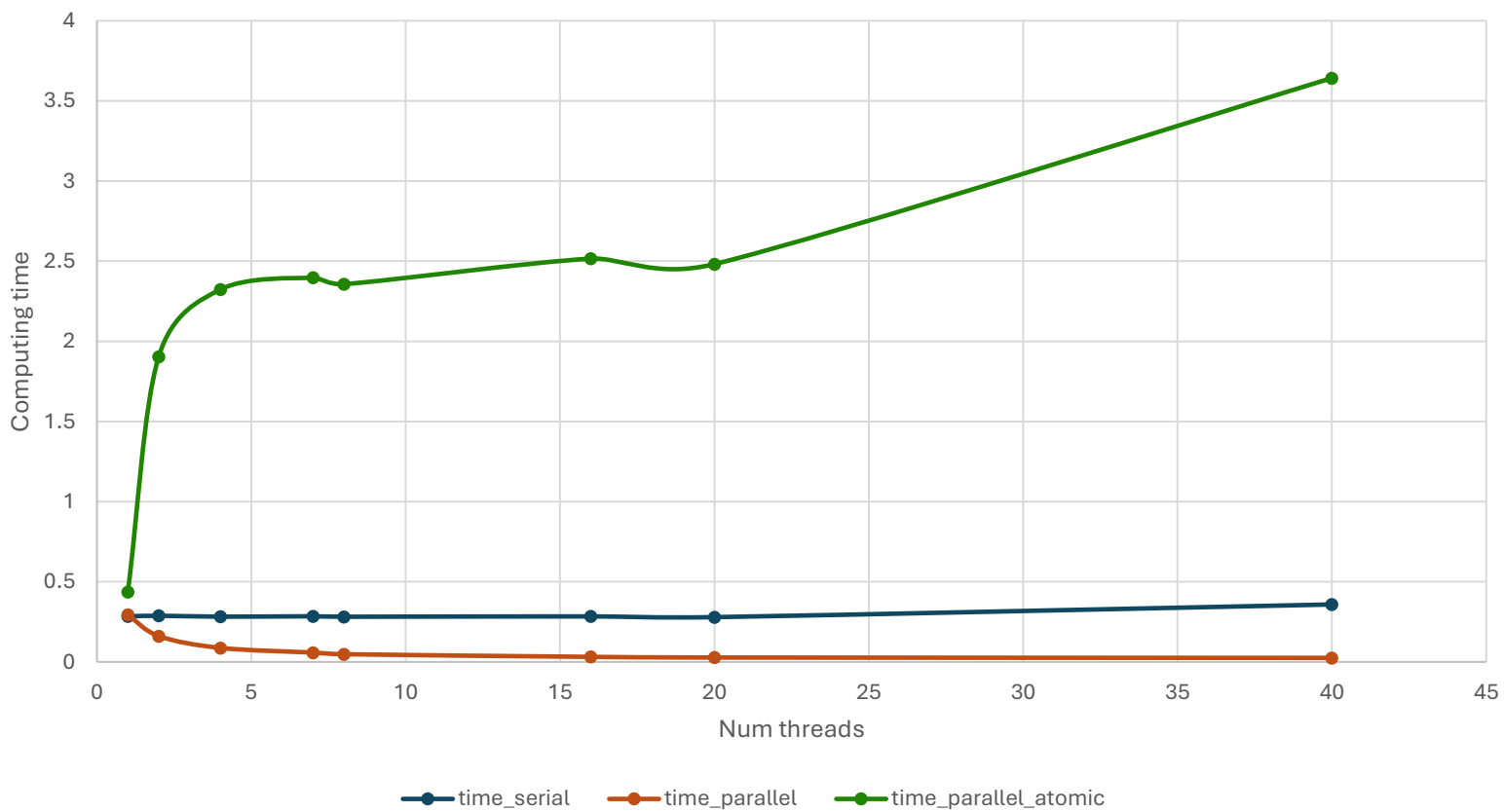
Speedup summary



Efficiency (spread)



Computing time vs thread amount



Вывод

Подход с использованием операции `atomic` значительно ухудшает производительность, предположительно, потому что каждое обращение к разделяемой переменной превращается в сериализованный, дорогой с точки зрения кеш-согласования доступ, из-за чего потоки простаивают в ожидании освобождения линии кеша. При большом числе потоков это может приводить к росту времени ожидания, высоким накладным расходам на протокол когерентности кеша между NUMA-нодами и, возможно, ложному разделению (`false sharing`), поэтому масштабирование практически пропадает.

Все варианты привязки потоков показали схожий прирост производительности, но наиболее линейный прирост дала политика `spread`, вероятно, потому что она равномерно распределяет потоки по всем доступным ядрам и NUMA-нодам, позволяя лучше использовать суммарное количество ядер и пропускную способность памяти. При такой схеме уменьшается конкуренция потоков за одни и те же вычислительные и кеш-ресурсы внутри сокета, снижается внутрисокетное вытеснение кеша, а межнодовый доступ к памяти становится более предсказуемым, что в итоге даёт более близкое к линейному масштабирование.